

# Clean Code moderne

pour React, TypeScript et Next.js

Cours progressif pour transformer un projet fonctionnel mais difficile à lire

## Objectif concret

Savoir décider comment nommer, découper, placer et faire évoluer le code d'une application Next.js exportée en site statique qui consomme un backend par API.

Adaptation critique de *Clean Code: A Handbook of Agile Software Craftsmanship*, deuxième édition

Document A4 – version du 23 juillet 2026

## Table des matières

|           |                                                                 |           |
|-----------|-----------------------------------------------------------------|-----------|
| <b>1</b>  | <b>Mode d'emploi du cours</b>                                   | <b>1</b>  |
| 1.1       | Comment apprendre avec ce document . . . . .                    | 1         |
| 1.2       | Ce qui vient du livre, ce qui est adapté . . . . .              | 1         |
| <b>2</b>  | <b>Carte critique de <i>Clean Code</i></b>                      | <b>1</b>  |
| 2.1       | Passages réellement utilisés . . . . .                          | 1         |
| 2.2       | Ce que nous conservons . . . . .                                | 2         |
| 2.3       | Ce que nous adaptons ou rejetons . . . . .                      | 3         |
| <b>3</b>  | <b>Lire le code comme un système d'explications</b>             | <b>3</b>  |
| 3.1       | Nommage explicite . . . . .                                     | 3         |
| 3.2       | Choisir la bonne longueur . . . . .                             | 4         |
| 3.3       | Noms de fichiers . . . . .                                      | 4         |
| <b>4</b>  | <b>Fonctions : rendre les décisions visibles</b>                | <b>5</b>  |
| 4.1       | Courtes, mais pour une raison . . . . .                         | 5         |
| 4.2       | La règle des 40 lignes . . . . .                                | 6         |
| 4.3       | Niveaux d'abstraction . . . . .                                 | 6         |
| 4.4       | Arguments et options . . . . .                                  | 7         |
| <b>5</b>  | <b>Responsabilités et duplication</b>                           | <b>8</b>  |
| 5.1       | Une responsabilité unique, concrètement . . . . .               | 8         |
| 5.2       | Reconnaître la duplication . . . . .                            | 8         |
| <b>6</b>  | <b>Commentaires : expliquer ce que le code ne peut pas dire</b> | <b>9</b>  |
| 6.1       | Le problème résolu . . . . .                                    | 9         |
| 6.2       | Quand commenter . . . . .                                       | 9         |
| 6.3       | Quand ne pas commenter . . . . .                                | 10        |
| <b>7</b>  | <b>Erreurs : des états explicites, pas des surprises</b>        | <b>10</b> |
| 7.1       | Le problème résolu . . . . .                                    | 10        |
| 7.2       | Modéliser les états asynchrones . . . . .                       | 11        |
| <b>8</b>  | <b>Composants React : séparer rendu, interaction et métier</b>  | <b>11</b> |
| 8.1       | Pourquoi les composants grossissent . . . . .                   | 12        |
| 8.2       | Mauvais exemple réaliste . . . . .                              | 12        |
| 8.3       | Découpage guidé par les responsabilités . . . . .               | 12        |
| 8.4       | Quand créer un composant . . . . .                              | 13        |
| 8.5       | Hooks : logique d'interface, pas dossier magique . . . . .      | 13        |
| <b>9</b>  | <b>Séparer UI, application, domaine et API</b>                  | <b>14</b> |
| 9.1       | Les quatre rôles . . . . .                                      | 14        |
| 9.2       | Direction des dépendances . . . . .                             | 14        |
| 9.3       | DTO et modèle de domaine . . . . .                              | 15        |
| <b>10</b> | <b>Architecture Next.js statique avec backend externe</b>       | <b>15</b> |
| 10.1      | Contrainte structurante : l'export statique . . . . .           | 15        |
| 10.2      | Arborescence proposée . . . . .                                 | 16        |
| 10.3      | Responsabilités des dossiers . . . . .                          | 16        |
| 10.4      | Méthode pour placer un fichier . . . . .                        | 17        |
| 10.5      | Imports et API publique d'une feature . . . . .                 | 17        |
| 10.6      | Constantes, types et utilitaires . . . . .                      | 18        |

|                                                                   |           |
|-------------------------------------------------------------------|-----------|
| <b>11 Cas complet : suivre une journée</b>                        | <b>18</b> |
| 11.1 Le comportement visé . . . . .                               | 18        |
| 11.2 Avant : un composant qui possède tout . . . . .              | 18        |
| 11.3 Diagnostic précis . . . . .                                  | 19        |
| 11.4 Après, étape 1 : types et domaine purs . . . . .             | 20        |
| 11.5 Après, étape 2 : frontière API . . . . .                     | 21        |
| 11.6 Après, étape 3 : hook d'orchestration . . . . .              | 22        |
| 11.7 Après, étape 4 : formulaire de présentation . . . . .        | 23        |
| 11.8 Après, étape 5 : page de composition . . . . .               | 25        |
| 11.9 Tests ciblés . . . . .                                       | 26        |
| <b>12 Détecter une complexité excessive</b>                       | <b>27</b> |
| 12.1 Tableau de signaux . . . . .                                 | 27        |
| 12.2 Complexité cognitive avant complexité cyclomatique . . . . . | 27        |
| <b>13 Refactoriser sans tout réécrire</b>                         | <b>28</b> |
| 13.1 Principe : préserver le comportement . . . . .               | 28        |
| 13.2 Méthode en huit étapes . . . . .                             | 28        |
| 13.3 Taille des lots . . . . .                                    | 28        |
| 13.4 Ordre de priorité . . . . .                                  | 28        |
| <b>14 Checklists de décision</b>                                  | <b>28</b> |
| 14.1 Avant de créer un composant . . . . .                        | 29        |
| 14.2 Avant de créer une fonction . . . . .                        | 29        |
| 14.3 Avant de créer un fichier . . . . .                          | 29        |
| 14.4 Avant une fusion . . . . .                                   | 29        |
| <b>15 Exercices de consolidation</b>                              | <b>29</b> |
| <b>16 Rappel actif et plan de progression</b>                     | <b>31</b> |
| 16.1 Questions à reconstruire sans le cours . . . . .             | 31        |
| 16.2 Calendrier de révision . . . . .                             | 31        |
| <b>17 Checklist finale de développement</b>                       | <b>31</b> |
| <b>18 Ce qu'une analyse du dépôt réel doit confirmer</b>          | <b>32</b> |
| <b>19 Références utilisées</b>                                    | <b>32</b> |

# 1 Mode d'emploi du cours

Ce document ne résume pas mécaniquement le livre. Il sélectionne les idées qui aident un développeur React/TypeScript à réduire trois coûts :

1. le temps nécessaire pour comprendre un fichier ;
2. le risque de casser un comportement lors d'une modification ;
3. l'effort nécessaire pour trouver le bon endroit où ajouter du code.

Le contexte supposé est réaliste : l'application fonctionne, elle a grandi par ajouts successifs, et la structure n'est plus évidente. Ce n'est pas un échec. C'est le moment où le développement change de nature : il ne suffit plus de produire un résultat, il faut rendre le prochain changement prévisible.

## 1.1 Comment apprendre avec ce document

Pour chaque notion, la séquence reste volontairement stable :

1. reconnaître le problème ;
2. comprendre le principe ;
3. lire un exemple dégradé ;
4. expliquer précisément le coût caché ;
5. étudier une version améliorée ;
6. reconstruire la règle sans regarder la correction.

Les exercices sont courts. Il vaut mieux les refaire de mémoire le lendemain que les lire passivement une seule fois.

### Réflexe pratique

Après chaque chapitre, fermez le PDF et formulez une règle avec vos propres mots. Si vous ne pouvez pas produire un exemple personnel, la notion n'est pas encore utilisable.

## 1.2 Ce qui vient du livre, ce qui est adapté

| Catégorie                 | Traitement dans ce cours                                                                                                                                                                                                           |
|---------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Principe durable</b>   | Conservé lorsqu'il améliore la compréhension, le test ou la capacité de changement : noms explicites, une responsabilité cohérente, séparation des niveaux d'abstraction, erreurs visibles, dépendances orientées vers le domaine. |
| <b>Conseil Java</b>       | Traduit en fonctions, composants, hooks, modules TypeScript et limites d'architecture. Les classes ne sont pas utilisées par défaut.                                                                                               |
| <b>Règle discutable</b>   | Présentée comme heuristique. Par exemple, une petite fonction n'est pas automatiquement bonne et une couverture de tests de 100 % n'est pas un objectif universel.                                                                 |
| <b>Contrainte moderne</b> | Ajoutée lorsqu'elle est propre à React ou Next.js : rendu, état, effets, frontière client, export statique, validation des réponses API et imports.                                                                                |

# 2 Carte critique de *Clean Code*

## 2.1 Passages réellement utilisés

Les pages ci-dessous désignent la pagination du fichier PDF fourni. Les titres de chapitres sont les repères les plus robustes entre éditions numériques.

| Chapitre ou passage                          | Pages PDF      | Utilité pour le projet                                                                                  |
|----------------------------------------------|----------------|---------------------------------------------------------------------------------------------------------|
| Chap. 1, <i>Clean Code</i>                   | début du livre | Lisibilité comme propriété de maintenance, pas comme décoration.                                        |
| Chap. 4, <i>Meaningful Names</i>             | 140–147        | Noms qui révèlent l'intention, évitent les fausses pistes et distinguent les concepts.                  |
| Chap. 5, <i>Comments</i>                     | 161–172        | Commentaires qui expliquent une décision ; rejet des commentaires qui répètent ou contredisent le code. |
| Chap. 6, <i>Formatting</i>                   | 198–200        | Cohérence locale, proximité des éléments liés et fichiers lisibles verticalement.                       |
| Chap. 7, <i>Clean Functions</i>              | 228–249        | Petites unités, séparation des niveaux d'abstraction, lecture descendante.                              |
| Chap. 8, <i>Function Heuristics</i>          | 259–269        | Arguments, séparation commande/requête, exceptions et erreur comme responsabilité.                      |
| Chap. 10, <i>One Thing</i>                   | 334–336        | Extraction guidée par une raison de changer, pas par un quota de lignes.                                |
| Chap. 12, <i>Objects and Data Structures</i> | 399–404        | Éviter les chaînes de connaissance et utiliser des objets de transfert aux frontières.                  |
| Chap. 13, <i>Clean Classes</i>               | 419–420        | Cohésion et responsabilité unique, adaptées ici aux modules et composants.                              |
| Chap. 14–15, tests                           | 466–480        | Tests lisibles, rapides, indépendants et ciblés sur le comportement.                                    |
| Chap. 18, <i>Simple Design</i>               | 513–521        | YAGNI, expression de l'intention, duplication et petite taille du système.                              |
| Chap. 19, SOLID                              | 523–528        | Responsabilité unique et dépendances, à appliquer sans fabriquer une forêt de classes.                  |
| Chap. 24–27, architecture                    | 691–724        | Frontière UI/application, isolation des systèmes externes et règle de dépendance.                       |

## 2.2 Ce que nous conservons

- Le code est lu beaucoup plus souvent qu'il n'est écrit.
- Un nom doit réduire le besoin de contexte caché.
- Une unité cohérente doit avoir une raison principale de changer.
- Les détails d'infrastructure ne doivent pas envahir les règles métier.
- Les tests donnent le courage de modifier une structure existante.
- La duplication est un signal : deux morceaux de code peuvent exprimer le même concept absent.

## 2.3 Ce que nous adaptons ou rejetons

| Prescription                          | Limite                                                                                                            | Adaptation moderne                                                                                       |
|---------------------------------------|-------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------|
| Fonctions extrêmement petites         | Un découpage excessif force à naviguer entre dix noms pour comprendre une opération simple.                       | Découper lorsqu'une fonction mélange des décisions, des effets ou des niveaux d'abstraction.             |
| Une seule chose                       | La formule est abstraite si l'on ne définit pas le niveau.                                                        | Une responsabilité cohérente et une raison principale de changer.                                        |
| Commentaires presque toujours mauvais | Les contraintes métier, compromis, formats externes et raisons non évidentes ne vivent pas toujours dans le code. | Commenter le <i>pourquoi</i> , le risque ou le contrat externe ; supprimer la narration du <i>quoi</i> . |
| Couverture proche de 100 %            | Le pourcentage mesure des lignes exécutées, pas la pertinence des assertions.                                     | Tester d'abord les règles métier, les transformations, les erreurs et les parcours critiques.            |
| Orientation classes                   | React et TypeScript favorisent souvent les fonctions et modules.                                                  | Appliquer cohésion et dépendances aux fonctions, hooks, composants et modules.                           |

### À retenir

*Clean Code* fournit des questions de conception, pas une collection de commandements. Une règle est utile si elle rend le comportement plus visible et le changement plus local.

## 3 Lire le code comme un système d'explications

### 3.1 Nommage explicite

#### 3.1.1 Le problème résolu

Un nom vague déplace l'information dans la tête du lecteur. `data`, `item`, `res`, `handle` ou `process` obligent à lire l'implémentation pour savoir ce qui existe, dans quelle unité et dans quel état.

Source : *Clean Code*, 2e éd., chap. 4, "Use Intention-Revealing Names", p. 140–147.

#### 3.1.2 Exemple insuffisant

```
const d = 7;

function handle(data: any) {
  const x = data.filter((item: any) => item.s > 5);
  return x;
}
```

Listing 1: Noms génériques et unités cachées

Ce code cache au moins cinq informations :

- `d` représente peut-être un nombre de jours ;
- `data` contient peut-être des entrées de journée ;
- `s` est peut-être un score ;
- `5` est un seuil métier ;
- `handle` ne précise ni l'action ni le résultat.

### 3.1.3 Version améliorée

```
type DayEntry = {
  date: string;
  energyScore: number;
};

const RECENT_PERIOD_DAYS = 7;
const HIGH_ENERGY_THRESHOLD = 5;

function selectHighEnergyDays(entries: DayEntry[]): DayEntry[] {
  return entries.filter(
    (entry) => entry.energyScore > HIGH_ENERGY_THRESHOLD,
  );
}
```

Listing 2: Le vocabulaire révèle le domaine

Les noms ne décrivent pas la syntaxe. Ils décrivent le domaine : journée, énergie, seuil et sélection.

#### Réflexe pratique

Pour nommer une fonction, utilisez un verbe précis et son résultat : `parseDayEntryResponse`, `calculateWeeklyAverage`, `selectEntriesInRange`. Évitez `manage`, `handle` et `process` sauf si le contexte donne déjà une signification exacte.

## 3.2 Choisir la bonne longueur

Un nom long n'est pas automatiquement clair. Sa longueur doit compenser le manque de contexte.

- Dans une callback de trois lignes, `entry` suffit.
- Dans un module partagé, `selectedEntryDate` est plus sûr que `date`.
- Une unité doit apparaître dans le nom : `durationMinutes`, `delayMs`, `distanceKm`.
- Un booléen doit se lire comme une question : `isSaving`, `hasEntries`, `canEditEntry`.

#### Erreur fréquente

Ajouter des suffixes comme `Data`, `Info`, `Object`, `Manager` ou `Helper` sans différence conceptuelle. Si `DayData` et `DayInfo` coexistent, leur distinction doit être évidente dans leur nom.

## 3.3 Noms de fichiers

Adoptez une convention stable :

| Type de fichier | Exemple et intention                                                                               |
|-----------------|----------------------------------------------------------------------------------------------------|
| Composant React | <code>DayEntryForm.tsx</code> : export principal du même nom.                                      |
| Hook            | <code>useDayEntries.ts</code> : préfixe <code>use</code> , responsabilité identifiable.            |
| Accès API       | <code>dayEntriesApi.ts</code> : opérations HTTP concernant les entrées.                            |
| Règle métier    | <code>dayScore.ts</code> ou <code>calculateDayScore.ts</code> : concept pur, sans React ni réseau. |
| Types locaux    | <code>types.ts</code> uniquement dans une feature ; sinon un nom de domaine plus précis.           |
| Test            | <code>calculateDayScore.test.ts</code> placé près du code testé.                                   |

### Exercice 1 – Renommer

Renommez les éléments suivants sans voir leur implémentation : `handle`, `data2`, `flag`, `time`, sachant qu'ils servent respectivement à enregistrer une entrée, contenir les entrées filtrées, indiquer un enregistrement en cours et mesurer des minutes de sommeil.

### Correction 1

`saveDayEntry`, `filteredEntries`, `isSaving`, `sleepDurationMinutes`. Plusieurs réponses sont valides si elles révèlent la même intention et restent cohérentes avec le vocabulaire du projet.

## 4 Fonctions : rendre les décisions visibles

### 4.1 Courtes, mais pour une raison

Le livre recommande de petites fonctions et une lecture descendante. L'idée durable n'est pas un nombre magique : une fonction doit rester à un niveau d'abstraction compréhensible.

Source : *Clean Code*, 2e éd., chap. 7, "Small!" et "The Stepdown Rule", p. 228–249 ; chap. 10, "Extract Method Refactoring", p. 334–336.

#### 4.1.1 Exemple trop chargé

```
async function save(entry: DayEntryFormValues) {
  if (!entry.date || entry.energy < 1 || entry.energy > 10) {
    setError('Invalid entry');
    return;
  }

  const score = Math.round(
    (entry.energy * 0.5 +
     entry.focus * 0.3 +
     Math.min(entry.sleepHours, 8) * 0.2) * 10,
  ) / 10;

  setIsSaving(true);
  try {
    const response = await fetch(`${API_URL}/entries`, {
      method: 'POST',
      headers: { 'Content-Type': 'application/json' },
      body: JSON.stringify({ ...entry, score }),
    });

    if (!response.ok) {
      throw new Error(`HTTP ${response.status}`);
    }

    const created = await response.json();
    setEntries((current) => [created, ...current]);
    setSuccess('Entry saved');
  } catch (error) {
    console.error(error);
    setError('Unable to save');
  } finally {
    setIsSaving(false);
  }
}
```

Listing 3: Validation, calcul, stockage et affichage mélangés

Le problème n'est pas seulement la longueur. Cette fonction change si :

- les règles de validation changent ;
- la formule de score change ;
- le contrat HTTP change ;
- la stratégie d'état React change ;
- le texte de l'interface change.

#### 4.1.2 Version améliorée

```

async function submitDayEntry(values: DayEntryFormValues) {
  const validation = validateDayEntry(values);

  if (!validation.ok) {
    setFormErrors(validation.errors);
    return;
  }

  setSaveState({ status: 'saving' });

  try {
    const entry = toDayEntry(values);
    const createdEntry = await dayEntriesApi.create(entry);

    setEntries((current) => [createdEntry, ...current]);
    setSaveState({ status: 'success' });
  } catch (error: unknown) {
    setSaveState({
      status: 'error',
      message: getUserFacingError(error),
    });
  }
}

```

Listing 4: Un flux principal lisible au même niveau

Le flux décrit une histoire : valider, signaler le chargement, transformer, envoyer, mettre à jour ou signaler l'erreur. Les détails sont placés dans les unités qui les possèdent.

## 4.2 La règle des 40 lignes

### Lecture moderne : les 40 lignes

Quarante lignes constituent un **signal d'alerte pratique**, jamais une loi. À partir de cette taille, posez quatre questions :

1. La fonction contient-elle plusieurs décisions métier ?
2. Mélange-t-elle calcul pur, réseau, stockage et interface ?
3. Faut-il faire défiler l'écran pour comprendre son résultat ?
4. Un bloc interne mérite-t-il un nom qui expliquerait son intention ?

Une fonction de 55 lignes peut être cohérente. Une fonction de 12 lignes peut déjà mélanger trois responsabilités.

## 4.3 Niveaux d'abstraction

Ce mélange est coûteux :

```

function buildWeeklySummary(entries: DayEntry[]) {
  const recent = entries.filter((entry) => {
    const timestamp = new Date(entry.date).getTime();
    return timestamp >= Date.now() - 7 * 24 * 60 * 60 * 1000;
  });
}

```

```
const average =
  recent.reduce((sum, entry) => sum + entry.energyScore, 0) /
  recent.length;

return {
  title: `Average: ${average.toFixed(1)}`,
  className: average >= 7 ? 'text-green-600' : 'text-orange-600',
};
}
```

La sélection temporelle, le calcul métier et la présentation CSS vivent au même endroit. Une séparation plus stable :

```
function buildWeeklySummary(entries: DayEntry[]): WeeklySummary {
  const weeklyEntries = selectEntriesFromLastDays(entries, 7);
  const averageEnergy = calculateAverageEnergy(weeklyEntries);

  return {
    averageEnergy,
    level: classifyEnergyLevel(averageEnergy),
  };
}
```

La composante React décide ensuite comment afficher `level`. Le domaine ne connaît pas Tailwind, une couleur ou une phrase traduite.

#### 4.4 Arguments et options

Plus une fonction possède de paramètres primitifs, plus l'appel devient illisible :

```
createEntry('2026-07-23', 8, 6, 7.5, true, '');
```

Préférez un objet lorsque les valeurs forment un concept :

```
createDayEntry({
  date: '2026-07-23',
  energyScore: 8,
  focusScore: 6,
  sleepHours: 7.5,
  exercised: true,
  note: '',
});
```

L'objet rend l'appel auto-documenté et facilite l'ajout d'un champ. Il ne faut pas pour autant transformer chaque paire de paramètres en type global.

#### À retenir

Découpez lorsqu'un nom extrait clarifie une décision, lorsqu'une partie peut être testée seule, ou lorsqu'un détail change pour une autre raison. Ne découpez pas uniquement pour satisfaire une métrique de lignes.

#### Exercice 2 – Niveau d'abstraction

Dans une fonction `loadDashboard`, repérez ce qui devrait être extrait : construction d'une URL, appel `fetch`, parsing JSON, calcul d'une moyenne, mise à jour d'un état React et choix d'une classe CSS.

**Correction 2**

L'accès HTTP et la validation de réponse appartiennent à un module API ; le calcul de moyenne au domaine ; l'état de chargement à un hook ou composant conteneur ; la classe CSS au composant de présentation. La fonction principale peut orchestrer ces unités, mais ne doit pas implémenter tous leurs détails.

## 5 Responsabilités et duplication

### 5.1 Une responsabilité unique, concrètement

La responsabilité unique ne signifie pas “un fichier ne fait qu'une ligne”. Elle signifie qu'une unité regroupe des éléments qui changent ensemble pour une raison cohérente.

Source : *Clean Code*, 2e éd., chap. 13, p. 419–420 ; chap. 19, “SRP”, p. 523–528.

| Unité                          | Responsabilité cohérente                                                                      |
|--------------------------------|-----------------------------------------------------------------------------------------------|
| <code>DayEntryForm.tsx</code>  | Collecter des valeurs, afficher les erreurs de formulaire, déclencher <code>onSubmit</code> . |
| <code>useDayEntries.ts</code>  | Exposer l'état de chargement et les actions nécessaires à l'interface.                        |
| <code>dayEntriesApi.ts</code>  | Traduire les opérations de l'application en requêtes HTTP et valider les réponses.            |
| <code>dayScore.ts</code>       | Calculer et classer un score sans dépendre de React ni du réseau.                             |
| <code>dayEntrySchema.ts</code> | Vérifier la forme des données reçues à une frontière non fiable.                              |

**Erreur fréquente**

Créer `utils.ts`, `helpers.ts` ou `common.ts` comme destination par défaut. Ces fichiers deviennent des tiroirs sans responsabilité commune et attirent des dépendances depuis tout le projet.

### 5.2 Reconnaître la duplication

La duplication n'est pas seulement deux blocs identiques. Elle peut être :

- une formule métier répétée dans deux composants ;
- le même `fetch` et le même traitement d'erreur dans plusieurs pages ;
- trois types presque identiques qui représentent en réalité des étapes différentes du flux ;
- plusieurs conditions qui définissent chacune ce qu'est une “bonne journée”.

Source : *Clean Code*, 2e éd., chap. 18, “Minimize Duplication”, p. 521.

#### 5.2.1 Mauvaise abstraction

```
function request(url: string, method: string, body?: any) {
  // Generic wrapper used by every possible endpoint.
}
```

Cette abstraction est créée trop tôt. Elle cache les différences et introduit `any`. Une meilleure première étape est une fonction précise :

```
async function createDayEntry(
  input: CreateDayEntryInput,
): Promise<DayEntry> {
  return httpClient.post('/day-entries', input, dayEntrySchema);
}
```

Le petit client HTTP partagé peut apparaître plus tard, après deux ou trois usages réellement similaires.

### Lecture moderne : DRY et YAGNI

**DRY** ne signifie pas “aucune ligne répétée”. Extraire trop tôt peut coupler des comportements qui évolueront différemment. Attendez de comprendre le concept commun. **YAGNI** rappelle de ne pas maintenir aujourd’hui une flexibilité hypothétique coûteuse.

### Réflexe pratique

Avant d’extraire du code dupliqué, terminez cette phrase : “Ces blocs représentent tous les deux le concept de ...”. Si vous ne pouvez pas nommer le concept, l’abstraction est probablement prématurée.

## 6 Commentaires : expliquer ce que le code ne peut pas dire

### 6.1 Le problème résolu

Les commentaires vieillissent séparément du code. Un commentaire narratif ajoute une deuxième version du comportement, souvent moins fiable.

Source : *Clean Code*, 2e éd., chap. 5, “Compensating for Failure”, “Good Comments” et “Bad Comments”, p. 161–172.

#### 6.1.1 Commentaire inutile

```
// Filter the entries to keep entries with energy above 7.
const goodEntries = entries.filter(
  (entry) => entry.energyScore > 7,
);
```

Le commentaire répète la syntaxe et le nom `goodEntries` reste vague. La règle métier mérite un nom :

```
const HIGH_ENERGY_THRESHOLD = 7;

const highEnergyEntries = entries.filter(
  ({ energyScore }) => energyScore >= HIGH_ENERGY_THRESHOLD,
);
```

#### 6.1.2 Commentaire utile

```
// The backend stores dates without a timezone. Append midnight UTC
// here to avoid displaying the previous day west of UTC.
const entryDate = new Date(`${apiEntry.date}T00:00:00Z`);
```

Ce commentaire explique une contrainte externe et le risque évité. Il doit idéalement être accompagné d’un test, car la gestion des dates reste fragile.

### 6.2 Quand commenter

- Une décision de conception surprenante et sa raison.
- Un contournement temporaire lié à une version ou un bug externe, avec un lien stable ou un identifiant de tâche.
- Une contrainte de sécurité, de performance ou de compatibilité.

- Le sens d’une expression régulière ou d’un format non évident.
- Le contrat public d’une bibliothèque lorsque les types ne suffisent pas.

### 6.3 Quand ne pas commenter

- Pour paraphraser chaque instruction.
- Pour compenser un nom vague.
- Pour conserver du code mort : Git possède déjà l’historique.
- Pour afficher un journal de modifications dans le fichier.
- Pour promettre un comportement que les tests ne vérifient pas.

#### À retenir

Le code explique *quoi* et *comment*. Un commentaire utile explique principalement *pourquoi* *cette décision existe*, quelle contrainte externe l’impose ou quel risque serait invisible.

## 7 Erreurs : des états explicites, pas des surprises

### 7.1 Le problème résolu

Les appels réseau échouent. Une réponse HTTP peut être un succès technique mais contenir une forme inattendue. Un `catch` qui ignore l’erreur transforme un défaut clair en état incohérent.

Source : *Clean Code*, 2e éd., chap. 8, “Prefer Exceptions to Returning Error Codes” et “Error Handling Is One Thing”, p. 266–269.

#### 7.1.1 Exemple dangereux

```
async function getEntries() {
  try {
    const response = await fetch(`${API_URL}/entries`);
    return await response.json();
  } catch {
    return [];
  }
}
```

Une panne serveur devient une liste vide. L’interface peut annoncer “aucune entrée” alors que les données sont simplement indisponibles. De plus, `response.ok` et la forme JSON ne sont jamais vérifiées.

#### 7.1.2 Version améliorée

```
class ApiError extends Error {
  constructor(
    message: string,
    readonly status?: number,
    readonly cause?: unknown,
  ) {
    super(message);
    this.name = 'ApiError';
  }
}

async function listDayEntries(): Promise<DayEntry[]> {
  let response: Response;

  try {
```

```

    response = await fetch(`${API_BASE_URL}/day-entries`);
  } catch (cause: unknown) {
    throw new ApiError('Network request failed', undefined, cause);
  }

  if (!response.ok) {
    throw new ApiError(
      'Unable to load day entries',
      response.status,
    );
  }

  const payload: unknown = await response.json();
  return parseDayEntries(payload);
}

```

La frontière API produit soit une donnée validée, soit une erreur explicite. Le hook choisit ensuite le message utilisateur et la stratégie de nouvelle tentative.

## 7.2 Modéliser les états asynchrones

Deux booléens indépendants permettent des combinaisons impossibles comme `isLoading === true` et `hasError === true`. Une union discriminée rend les états autorisés explicites :

```

type LoadState<T> =
  | { status: 'idle' }
  | { status: 'loading' }
  | { status: 'success'; data: T }
  | { status: 'error'; message: string };

```

Le rendu devient exhaustif :

```

switch (entriesState.status) {
  case 'idle':
  case 'loading':
    return <DayEntriesSkeleton />;
  case 'error':
    return <LoadError message={entriesState.message} />;
  case 'success':
    return <DayEntryList entries={entriesState.data} />;
}

```

### Erreur fréquente

Utiliser `any` à la frontière réseau. TypeScript ne vérifie pas le JSON à l'exécution. Une annotation `const data: DayEntry[]` ne transforme pas une réponse inconnue en donnée fiable. Parsez `unknown` avec un schéma ou une fonction de validation.

### Réflexe pratique

À chaque appel externe, définissez séparément : succès, erreur HTTP, erreur réseau, données invalides et annulation éventuelle. L'interface ne doit jamais confondre “vide” et “indisponible”.

## 8 Composants React : séparer rendu, interaction et métier

## 8.1 Pourquoi les composants grossissent

Un composant commence souvent avec un formulaire et une liste. Puis arrivent le chargement, la validation, le calcul, les filtres, les modales, le réseau et les notifications. Le JSX n'est pas le problème : c'est le mélange des raisons de changer.

### 8.1.1 Signaux d'un composant trop volumineux

- plus de cinq ou six états locaux qui se coordonnent ;
- plusieurs `useEffect` dépendants les uns des autres ;
- des appels `fetch` au milieu du composant ;
- des calculs métier dans le JSX ;
- des sections visuelles nommables et indépendantes ;
- un test qui doit simuler tout l'écran pour vérifier une formule ;
- un tableau de dépendances `useEffect` difficile à justifier ;
- plus de 40 lignes par fonction de rendu ou callback comme signal d'examen, non comme faute automatique.

## 8.2 Mauvais exemple réaliste

```
export function Dashboard() {
  const [entries, setEntries] = useState<DayEntry[]>([]);
  const [loading, setLoading] = useState(false);
  const [error, setError] = useState('');
  const [filter, setFilter] = useState('all');

  useEffect(() => {
    setLoading(true);
    fetch(`${process.env.NEXT_PUBLIC_API_URL}/entries`)
      .then((response) => response.json())
      .then(setEntries)
      .catch(() => setError('Error'))
      .finally(() => setLoading(false));
  }, []);

  const average =
    entries.reduce((sum, entry) => sum + entry.energy, 0) /
    entries.length;

  return (
    <main>
      <h1>My days</h1>
      <p className={average > 7 ? 'green' : 'orange'}>
        {average.toFixed(1)}
      </p>
      {/* form, filters, list, modal and save logic ... */}
    </main>
  );
}
```

Listing 5: Le composant connaît trop de détails

Ce composant dépend du contrat du backend, d'une formule, d'une convention CSS et de la navigation de l'écran. Une panne renvoie peut-être une liste vide, et la moyenne d'une liste vide devient NaN.

## 8.3 Découpage guidé par les responsabilités

```
'use client';

export default function DayTrackingPage() {
```

```

const dayEntries = useDayEntries();
const summary = buildDaySummary(dayEntries.entries);

return (
  <DayTrackingLayout
    summary={<DaySummary summary={summary} />}
    form={
      <DayEntryForm
        isSaving={dayEntries.isSaving}
        onSubmit={dayEntries.createEntry}
      />
    }
    content={
      <DayEntryList
        entries={dayEntries.entries}
        status={dayEntries.loadStatus}
        onRetry={dayEntries.reload}
      />
    }
  />
);
}

```

Listing 6: Le composant de page compose des unités explicites

Le composant de page orchestre l'écran. Il ne connaît pas la forme HTTP, la formule du résumé ni les détails internes du formulaire.

## 8.4 Quand créer un composant

Créez un nouveau composant lorsqu'au moins une condition est vraie :

- la partie possède un nom métier ou visuel clair ;
- elle a des props simples et un comportement testable ;
- elle est réutilisée réellement ;
- elle masque un détail visuel encombrant ;
- elle possède son propre état local cohérent.

Ne créez pas un fichier pour un div et deux classes uniquement afin de réduire la taille apparente du parent.

## 8.5 Hooks : logique d'interface, pas dossier magique

Un hook personnalisé est pertinent lorsqu'il encapsule une mécanique React réutilisable ou complexe : synchronisation, état asynchrone, abonnement, coordination d'actions.

```

type UseDayEntriesResult = {
  entries: DayEntry[];
  loadStatus: 'loading' | 'ready' | 'error';
  isSaving: boolean;
  createEntry(input: CreateDayEntryInput): Promise<void>;
  reload(): Promise<void>;
};

export function useDayEntries(): UseDayEntriesResult {
  // React state and orchestration live here.
}

```

Une formule pure n'a pas besoin d'être un hook :

```
// Good: usable in React, Node, tests and scripts.
```

```
export function calculateAverageEnergy(
  entries: DayEntry[],
): number | null {
  if (entries.length === 0) return null;

  const total = entries.reduce(
    (sum, entry) => sum + entry.energyScore,
    0,
  );
  return total / entries.length;
}
```

### Lecture moderne : hooks et domaine

Le hook appartient à la couche présentation/application parce qu'il dépend du cycle de vie React. Les règles métier doivent rester dans des fonctions pures lorsqu'elles n'ont pas besoin de React. Cela simplifie les tests et évite d'appeler des hooks pour réutiliser un calcul.

### Exercice 3 – Découper un composant

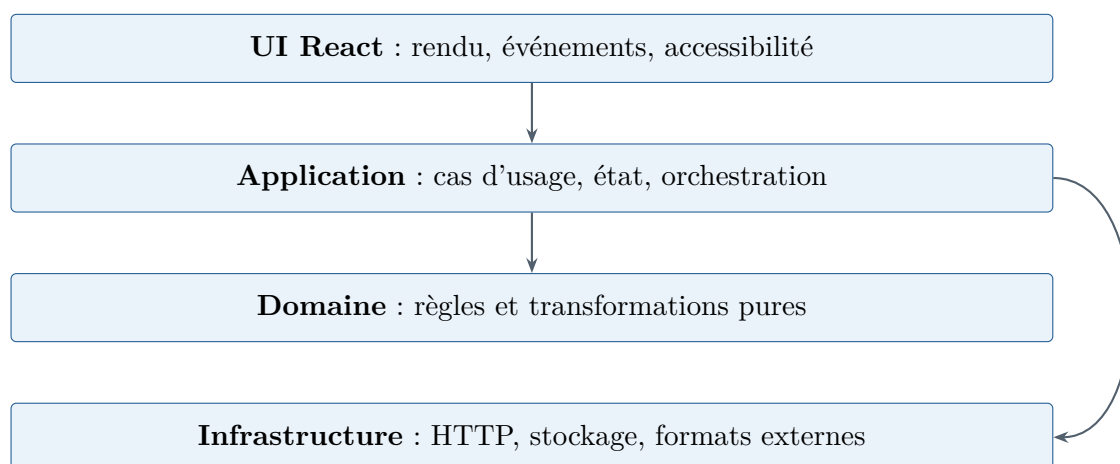
Un composant contient : champs de formulaire, validation métier, appel POST, notification de succès, calcul d'une série hebdomadaire et graphique. Proposez les unités minimales.

### Correction 3

DayEntryForm gère les champs et erreurs de saisie ; validateDayEntry porte les règles ; dayEntriesApi.create porte le POST ; useDayEntries orchestre l'état et la notification ; buildWeeklySeries calcule les points ; WeeklyEnergyChart affiche la série. Le découpage exact peut varier, mais le réseau et le domaine ne doivent pas être enfouis dans le graphique ou le formulaire.

## 9 Séparer UI, application, domaine et API

### 9.1 Les quatre rôles



Source : *Clean Code*, 2e éd., chap. 24–27, frontière UI/application et règle de dépendance, p. 691–724.

### 9.2 Direction des dépendances

Le domaine ne doit pas importer React, Next.js, `fetch`, le CSS ou une bibliothèque de notifications. L'infrastructure peut importer les types de domaine nécessaires pour traduire une réponse. L'UI appelle les cas d'usage exposés par les hooks ou services.

| Couche         | Peut connaître                                    | Ne devrait pas connaître                                       |
|----------------|---------------------------------------------------|----------------------------------------------------------------|
| UI             | props, composants, événements, modèles de vue     | URL backend, structure JSON brute, règles de calcul détaillées |
| Application    | cas d'usage, ports API, état d'opération          | classes CSS, structure interne d'un composant                  |
| Domaine        | types métier, invariants, calculs                 | React, navigateur, HTTP, variables d'environnement             |
| Infrastructure | <code>fetch</code> , schémas, endpoints, stockage | détails de rendu et décisions visuelles                        |

### 9.3 DTO et modèle de domaine

La réponse du backend n'est pas nécessairement le modèle utilisé par l'interface :

```
type DayEntryDto = {
  id: string;
  entry_date: string;
  energy_score: number;
  created_at: string;
};

type DayEntry = {
  id: string;
  date: LocalDate;
  energyScore: number;
  createdAt: Date;
};

function toDayEntry(dto: DayEntryDto): DayEntry {
  return {
    id: dto.id,
    date: parseLocalDate(dto.entry_date),
    energyScore: dto.energy_score,
    createdAt: new Date(dto.created_at),
  };
}
```

Cette traduction évite de répandre `snake_case`, les chaînes de dates et les choix du backend dans tous les composants.

#### À retenir

Une frontière propre transforme les données externes une seule fois. À l'intérieur de l'application, travaillez avec un vocabulaire et des types cohérents.

## 10 Architecture Next.js statique avec backend externe

### 10.1 Contrainte structurante : l'export statique

Avec un export statique, les fichiers HTML, JavaScript et CSS sont produits au build puis servis sans serveur Next.js permanent. Par conséquent :

- les routes API Next.js et Server Actions ne fournissent pas un backend disponible à l'exécution sur un simple hébergement statique ;
- les données privées ou fréquemment mises à jour sont chargées dans le navigateur depuis un backend externe ;
- seules les variables préfixées `NEXT_PUBLIC_` peuvent être intégrées au code client ; elles ne doivent contenir aucun secret ;

- le backend doit gérer CORS, authentification, validation et autorisation ;
- les fonctionnalités Next.js incompatibles avec l'export statique doivent être évitées ou remplacées.

### Erreur fréquente

Placer une clé secrète dans `NEXT_PUBLIC_API_KEY`. Le préfixe signifie que la valeur peut être livrée au navigateur. Une application statique ne peut pas protéger un secret côté client.

## 10.2 Arborescence proposée

```
src/
  app/
    layout.tsx          # shell global
    page.tsx            # route d'accueil
    tracking/
      page.tsx          # composition de l'ecran

  features/
    day-tracking/
      api/
        dayEntriesApi.ts  # endpoints et mapping DTO
        dayEntrySchema.ts # validation des reponses
      components/
        DayEntryForm.tsx  # saisie et erreurs visibles
        DayEntryList.tsx  # affichage de la collection
        DaySummary.tsx    # presentation du resume
      domain/
        dayEntry.ts       # types et invariants du domaine
        dayScore.ts       # calculs purs
      hooks/
        useDayEntries.ts  # etat React et cas d'usage UI
      index.ts            # API publique de la feature

  components/
    ui/
      Button.tsx          # primitives partagees
      Field.tsx
      Spinner.tsx

  lib/
    http/
      httpClient.ts        # mecanique HTTP transversale
      ApiError.ts
    env/
      publicEnv.ts         # lecture validee de la config

  constants/
    routes.ts              # constantes vraiment globales

  static/
    icons/                 # ressources servies telles quelles

  test/
    factories/
      dayEntryFactory.ts   # builders partages par les tests
```

Listing 7: Organisation par fonctionnalités et couches locales

## 10.3 Responsabilités des dossiers

| Dossier                                      | Responsabilité                                                                                                                                                                                                                   |
|----------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>app/</code><br><code>features/</code>  | Routes, layouts et composition des écrans. Peu de règles métier. Code organisé par capacité utilisateur. Une feature possède ses composants, hooks, domaine et accès aux données tant qu'ils ne sont pas véritablement partagés. |
| <code>components/ui/</code>                  | Primitives visuelles génériques : bouton, champ, boîte de dialogue. Elles ne connaissent pas les entrées de journée.                                                                                                             |
| <code>lib/</code>                            | Infrastructure transversale stable : client HTTP, configuration, formatage générique. Ce n'est pas un nouveau <code>utils/</code> .                                                                                              |
| <code>constants/</code>                      | Constantes globales rares. Une constante métier locale reste dans sa feature.                                                                                                                                                    |
| <code>static/</code> ou <code>public/</code> | Fichiers servis tels quels. Avec Next.js, <code>public/</code> est généralement le dossier conventionnel ; conserver <code>static/</code> seulement si le projet possède un pipeline explicite.                                  |
| <code>test/</code>                           | Infrastructure de test partagée. Les tests unitaires restent idéalement près des modules concernés.                                                                                                                              |

## 10.4 Méthode pour placer un fichier

1. **Quelle fonctionnalité utilisateur sert-il ?** S'il répond clairement "suivi de journée", commencez dans `features/day-tracking/`.
2. **Quel est son rôle ?** Rendu : `components/`. État React : `hooks/`. HTTP : `api/`. Règle pure : `domain/`.
3. **Est-il réellement partagé ?** Un deuxième usage concret ne suffit pas toujours. Déplacez vers `components/ui/` ou `lib/` lorsque l'API devient générique et stable.
4. **Qui a le droit de l'importer ?** Définissez une direction. Le domaine ne remonte pas vers l'UI.
5. **Le nom du dossier ajoute-t-il une information ?** Évitez les catégories vagues comme `misc/`, `common/` ou `helpers/`.

## 10.5 Imports et API publique d'une feature

Un fichier `features/day-tracking/index.ts` peut limiter ce que les autres features utilisent :

```
export { DaySummary } from './components/DaySummary';
export { useDayEntries } from './hooks/useDayEntries';
export type { DayEntry } from './domain/dayEntry';
```

À l'intérieur de la feature, les imports directs restent acceptables. Depuis l'extérieur, passer par cette API publique réduit les dépendances sur la structure interne.

### Erreur fréquente

Créer des fichiers `index.ts` à tous les niveaux. Les "barrels" peuvent masquer les dépendances, créer des cycles et alourdir le graphe de modules. Utilisez-les aux frontières publiques, pas comme automatisme.

## 10.6 Constantes, types et utilitaires

| Élément                  | Placement conseillé                           | Exemple                                        |
|--------------------------|-----------------------------------------------|------------------------------------------------|
| Constante locale         | près de son usage                             | MAX_NOTE_LENGTH dans la validation de l'entrée |
| Constante de feature     | dans le domaine de la feature                 | seuils de score                                |
| Constante globale        | constants/ si plusieurs features en dépendent | routes de navigation                           |
| Type de composant        | dans le même fichier                          | DaySummaryProps                                |
| Type de domaine          | domain/                                       | DayEntry                                       |
| DTO externe              | api/                                          | DayEntryDto                                    |
| Utilitaire pur local     | près de la feature                            | groupEntriesByWeek                             |
| Infrastructure générique | lib/                                          | httpClient                                     |

### À retenir

Placez un fichier au plus près du concept qui le possède. Ne le rendez global qu'après avoir observé une responsabilité réellement transversale.

## 11 Cas complet : suivre une journée

### 11.1 Le comportement visé

L'utilisateur ouvre l'écran, voit ses entrées, saisit une date, une énergie, une concentration et une durée de sommeil, puis enregistre. L'interface affiche un résumé et distingue clairement chargement, absence de données et erreur.

Cet exemple est volontairement générique. Il **n'affirme pas** que le dépôt réel possède ces champs, endpoints ou bibliothèques. Les noms devront être adaptés après lecture du code et du contrat backend réels.

### 11.2 Avant : un composant qui possède tout

```
'use client';

export default function Tracker() {
  const [data, setData] = useState<any[]>([]);
  const [date, setDate] = useState('');
  const [energy, setEnergy] = useState(5);
  const [focus, setFocus] = useState(5);
  const [sleep, setSleep] = useState(7);
  const [loading, setLoading] = useState(false);
  const [saving, setSaving] = useState(false);
  const [error, setError] = useState('');

  useEffect(() => {
    setLoading(true);
    fetch(`${process.env.NEXT_PUBLIC_API_URL}/entries`)
      .then((r) => r.json())
      .then((json) => setData(json))
      .catch(() => setError('Something went wrong'))
      .finally(() => setLoading(false));
  }, []);

  async function handleSave() {
    if (!date) {
      setError('Date required');
    }
  }
}
```

```

    return;
  }

  const score =
    Math.round(
      (energy * 0.5 + focus * 0.3 + Math.min(sleep, 8) * 0.2) *
        10,
    ) / 10;

  setSaving(true);
  try {
    const response = await fetch(
      `${process.env.NEXT_PUBLIC_API_URL}/entries`,
      {
        method: 'POST',
        headers: { 'Content-Type': 'application/json' },
        body: JSON.stringify({
          entry_date: date,
          energy_score: energy,
          focus_score: focus,
          sleep_hours: sleep,
          score,
        }),
      },
    );

    const created = await response.json();
    setData((current) => [created, ...current]);
  } catch {
    setError('Unable to save');
  } finally {
    setSaving(false);
  }
}

const average =
  data.reduce((sum, item) => sum + item.energy_score, 0) /
  data.length;

return (
  <main>
    <h1>Day tracker</h1>
    {loading && <p>Loading...</p>}
    {error && <p>{error}</p>}
    <p>Average: {average.toFixed(1)}</p>
    { /* inputs and list omitted */ }
    <button disabled={saving} onClick={handleSave}>
      Save
    </button>
  </main>
);
}

```

Listing 8: Version condensée d'un composant monolithique

### 11.3 Diagnostic précis

| Symptôme     | Risque                                                                     |
|--------------|----------------------------------------------------------------------------|
| any []       | Toutes les erreurs de contrat deviennent possibles et se propagent au JSX. |
| fetch direct | Chaque écran doit réinventer URL, statut HTTP, parsing et erreurs.         |

| Symptôme                                   | Risque                                                                |
|--------------------------------------------|-----------------------------------------------------------------------|
| Noms <code>data</code> , <code>item</code> | Le domaine est absent du vocabulaire.                                 |
| Formule dans le callback                   | Impossible de tester le score sans monter le composant.               |
| DTO utilisé dans l'UI                      | <code>energy_score</code> et le format backend contaminent le projet. |
| Booléens indépendants                      | Des états incohérents deviennent représentables.                      |
| Liste vide et panne confondues             | Le message utilisateur peut être faux.                                |
| Moyenne sans cas vide                      | NaN apparaît lorsque la collection est vide.                          |
| Réponse POST non vérifiée                  | Un statut 500 peut être parsé comme un succès.                        |

## 11.4 Après, étape 1 : types et domaine purs

`features/day-tracking/domain/dayEntry.ts`

```
export type DayEntry = {
  id: string;
  date: string;
  energyScore: number;
  focusScore: number;
  sleepHours: number;
  dayScore: number;
};

export type CreateDayEntryInput = Omit<DayEntry, 'id' | 'dayScore'>;
```

`features/day-tracking/domain/dayScore.ts`

```
import type { DayEntry } from './dayEntry';

type ScoreInput = Pick<
  DayEntry,
  'energyScore' | 'focusScore' | 'sleepHours'
>;

export function calculateDayScore(input: ScoreInput): number {
  const cappedSleepHours = Math.min(input.sleepHours, 8);
  const weightedScore =
    input.energyScore * 0.5 +
    input.focusScore * 0.3 +
    cappedSleepHours * 0.2;

  return Math.round(weightedScore * 10) / 10;
}

export function calculateAverageEnergy(
  entries: DayEntry[],
): number | null {
  if (entries.length === 0) return null;

  const total = entries.reduce(
    (sum, entry) => sum + entry.energyScore,
    0,
  );
  return total / entries.length;
}
```

Ces fonctions n'importent ni React ni l'API. Elles peuvent être testées en quelques millisecondes.

## 11.5 Après, étape 2 : frontière API

features/day-tracking/api/dayEntriesApi.ts

```
import type {
  CreateDayEntryInput,
  DayEntry,
} from '../domain/dayEntry';
import { calculateDayScore } from '../domain/dayScore';
import { ApiError } from '@lib/http/ApiError';
import { publicEnv } from '@lib/env/publicEnv';

type DayEntryDto = {
  id: string;
  entry_date: string;
  energy_score: number;
  focus_score: number;
  sleep_hours: number;
  day_score: number;
};

function isDayEntryDto(value: unknown): value is DayEntryDto {
  if (typeof value !== 'object' || value === null) return false;

  const candidate = value as Record<string, unknown>;
  return (
    typeof candidate.id === 'string' &&
    typeof candidate.entry_date === 'string' &&
    typeof candidate.energy_score === 'number' &&
    typeof candidate.focus_score === 'number' &&
    typeof candidate.sleep_hours === 'number' &&
    typeof candidate.day_score === 'number'
  );
}

function fromDto(dto: DayEntryDto): DayEntry {
  return {
    id: dto.id,
    date: dto.entry_date,
    energyScore: dto.energy_score,
    focusScore: dto.focus_score,
    sleepHours: dto.sleep_hours,
    dayScore: dto.day_score,
  };
}

async function parseResponse(response: Response): Promise<unknown> {
  if (!response.ok) {
    throw new ApiError(
      'Day entries request failed',
      response.status,
    );
  }
  return response.json();
}

export const dayEntriesApi = {
  async list(): Promise<DayEntry[]> {
    const response = await fetch(
      `${publicEnv.apiUrl}/day-entries`,
    );
    const payload = await parseResponse(response);

    if (!Array.isArray(payload) || !payload.every(isDayEntryDto)) {
      throw new ApiError('Invalid day entries response');
    }
  }
}
```

```

    }
    return payload.map(fromDto);
  },

  async create(input: CreateDayEntryInput): Promise<DayEntry> {
    const response = await fetch(
      `${publicEnv.apiUrl}/day-entries`,
      {
        method: 'POST',
        headers: { 'Content-Type': 'application/json' },
        body: JSON.stringify({
          entry_date: input.date,
          energy_score: input.energyScore,
          focus_score: input.focusScore,
          sleep_hours: input.sleepHours,
          day_score: calculateDayScore(input),
        }),
      },
    );
    const payload = await parseResponse(response);

    if (!isDayEntryDto(payload)) {
      throw new ApiError('Invalid created entry response');
    }
    return fromDto(payload);
  },
};

```

Dans un projet réel, une bibliothèque de schémas déjà installée peut remplacer le garde manuel. Le choix dépend des dépendances présentes ; il ne faut pas en ajouter une sans examiner le dépôt.

## 11.6 Après, étape 3 : hook d'orchestration

features/day-tracking/hooks/useDayEntries.ts

```

'use client';

import { useCallback, useEffect, useState } from 'react';
import { dayEntriesApi } from '../api/dayEntriesApi';
import type {
  CreateDayEntryInput,
  DayEntry,
} from '../domain/dayEntry';

type EntriesState =
  | { status: 'loading'; entries: DayEntry[] }
  | { status: 'ready'; entries: DayEntry[] }
  | { status: 'error'; entries: DayEntry[]; message: string };

export function useDayEntries() {
  const [state, setState] = useState<EntriesState>({
    status: 'loading',
    entries: [],
  });
  const [isSaving, setIsSaving] = useState(false);
  const [saveError, setSaveError] = useState<string | null>(null);

  const reload = useCallback(async () => {
    setState((current) => ({
      status: 'loading',
      entries: current.entries,
    }));
  });

  try {

```

```

    const entries = await dayEntriesApi.list();
    setState({ status: 'ready', entries });
  } catch {
    setState((current) => ({
      status: 'error',
      entries: current.entries,
      message: 'Impossible de charger les journees.',
    }));
  }
}, []);

useEffect(() => {
  void reload();
}, [reload]);

const createEntry = useCallback(
  async (input: CreateDayEntryInput): Promise<boolean> => {
    setIsSaving(true);
    setSaveError(null);
    try {
      const createdEntry = await dayEntriesApi.create(input);
      setState((current) => ({
        status: 'ready',
        entries: [createdEntry, ...current.entries],
      }));
      return true;
    } catch {
      setSaveError("Impossible d'enregistrer cette journee.");
      return false;
    } finally {
      setIsSaving(false);
    }
  },
  [],
);

return {
  ...state,
  isSaving,
  saveError,
  createEntry,
  reload,
};
}

```

Le hook expose séparément l'erreur de chargement et l'erreur d'enregistrement. Une bibliothèque de cache serveur déjà présente, comme TanStack Query ou SWR, peut remplacer cette mécanique. Il serait contre-productif de la réécrire si le dépôt en utilise déjà une.

## 11.7 Après, étape 4 : formulaire de présentation

features/day-tracking/components/DayEntryForm.tsx

```

'use client';

import { useState } from 'react';
import type { FormEvent } from 'react';
import type { CreateDayEntryInput } from '../domain/dayEntry';

type DayEntryFormProps = {
  isSaving: boolean;
  submitError: string | null;
  onSubmit(input: CreateDayEntryInput): Promise<boolean>;
};

```

```
};

const initialValues: CreateDayEntryInput = {
  date: '',
  energyScore: 5,
  focusScore: 5,
  sleepHours: 7,
};

type NumberFieldProps = {
  label: string;
  value: number;
  min?: number;
  max: number;
  step?: number;
  onChange(value: number): void;
};

function NumberField({
  label,
  value,
  min = 1,
  max,
  step = 1,
  onChange,
}: NumberFieldProps) {
  return (
    <label>
      {label}
      <input
        type="number"
        min={min}
        max={max}
        step={step}
        required
        value={value}
        onChange={(event) => {
          const nextValue = event.target.valueAsNumber;
          if (!Number.isNaN(nextValue)) onChange(nextValue);
        }}
      />
    </label>
  );
}

export function DayEntryForm({
  isSaving,
  submitError,
  onSubmit,
}: DayEntryFormProps) {
  const [values, setValues] = useState(initialValues);
  const [dateError, setDateError] = useState<string | null>(null);

  async function handleSubmit(event: FormEvent<HTMLFormElement>) {
    event.preventDefault();

    if (!values.date) {
      setDateError('Choisissez une date.');
```

```

return (
  <form onSubmit={handleSubmit}>
    <label>
      Date
      <input
        type="date"
        required
        value={values.date}
        onChange={(event) =>
          setValues({ ...values, date: event.target.value })
        }
      />
    </label>
    {dateError && <p role="alert">{dateError}</p>}

    <NumberField
      label="Energie"
      value={values.energyScore}
      max={10}
      onChange={(energyScore) =>
        setValues({ ...values, energyScore })
      }
    />
    <NumberField
      label="Concentration"
      value={values.focusScore}
      max={10}
      onChange={(focusScore) =>
        setValues({ ...values, focusScore })
      }
    />
    <NumberField
      label="Sommeil"
      value={values.sleepHours}
      min={0}
      max={24}
      step={0.5}
      onChange={(sleepHours) =>
        setValues({ ...values, sleepHours })
      }
    />

    {submitError && <p role="alert">{submitError}</p>}

    <button type="submit" disabled={isSaving}>
      {isSaving ? 'Enregistrement...' : 'Enregistrer'}
    </button>
  </form>
);
}

```

Les champs sont contrôlés, bornés et associés à un libellé. La validation serveur reste obligatoire : les contraintes HTML améliorent la saisie, mais ne constituent pas une frontière de sécurité.

## 11.8 Après, étape 5 : page de composition

app/tracking/page.tsx

```

'use client';

import { DayEntryForm } from
  '@/features/day-tracking/components/DayEntryForm';
import { DayEntryList } from

```

```

    '@/features/day-tracking/components/DayEntryList';
import { DaySummary } from
    '@/features/day-tracking/components/DaySummary';
import { calculateAverageEnergy } from
    '@/features/day-tracking/domain/dayScore';
import { useDayEntries } from
    '@/features/day-tracking/hooks/useDayEntries';

export default function TrackingPage() {
  const {
    entries,
    status,
    isSaving,
    saveError,
    createEntry,
    reload,
  } = useDayEntries();

  return (
    <main>
      <h1>Suivi de mes journées</h1>
      <DaySummary
        averageEnergy={calculateAverageEnergy(entries)}
      />
      <DayEntryForm
        isSaving={isSaving}
        submitError={saveError}
        onSubmit={createEntry}
      />
      <DayEntryList
        entries={entries}
        status={status}
        onRetry={reload}
      />
    </main>
  );
}

```

La page lit maintenant comme une table des matières de l'écran.

## 11.9 Tests ciblés

```

describe('calculateAverageEnergy', () => {
  it('returns null for an empty collection', () => {
    expect(calculateAverageEnergy([])).toBeNull();
  });

  it('calculates the average energy', () => {
    const entries = [
      dayEntryFactory({ energyScore: 6 }),
      dayEntryFactory({ energyScore: 8 }),
    ];

    expect(calculateAverageEnergy(entries)).toBe(7);
  });
});

```

Listing 9: Test de règle métier, rapide et sans React

```

it('rejects an invalid list response', async () => {
  mockFetchResponse([{ id: '1', energy_score: 'high' }]);

  await expect(dayEntriesApi.list()).rejects.toThrow(

```

```
'Invalid day entries response',
);
});
```

Listing 10: Test du contrat de la frontière API

Les noms exacts des outils de test doivent suivre le dépôt. Le principe est de tester les règles pures et les frontières instables avant de multiplier les tests de rendu.

## 12 Détecter une complexité excessive

### 12.1 Tableau de signaux

| Signal                                             | Question à poser                                 | Action probable                                                              |
|----------------------------------------------------|--------------------------------------------------|------------------------------------------------------------------------------|
| Fonction de plus de 40 lignes                      | Mélange-t-elle plusieurs niveaux ?               | Nommer et extraire une décision ou un effet cohérent.                        |
| Plus de trois paramètres primitifs                 | Forment-ils un concept ?                         | Introduire un objet d'entrée précis.                                         |
| Plus de deux niveaux d'imbrication                 | Les cas invalides peuvent-ils sortir tôt ?       | Utiliser des gardes ou une table de décision.                                |
| Nombreux booléens d'état                           | Des combinaisons sont-elles impossibles ?        | Employer une union discriminée ou un reducer.                                |
| <code>useEffect</code> qui modifie plusieurs états | Est-ce une synchronisation ou un calcul dérivé ? | Calculer pendant le rendu, extraire un hook ou utiliser un reducer.          |
| Imports depuis beaucoup de zones                   | Le module connaît-il trop de couches ?           | Déplacer l'orchestration ou réduire son API.                                 |
| Conditions métier répétées                         | Le même concept est-il défini plusieurs fois ?   | Créer une fonction de domaine nommée.                                        |
| Props transmises sur plusieurs niveaux             | Le propriétaire de l'état est-il bien placé ?    | Revoir la composition ; contexte seulement pour un besoin réellement global. |
| <code>utils.ts</code> en croissance                | Les fonctions changent-elles ensemble ?          | Les replacer par domaine ou infrastructure.                                  |
| Test nécessitant tout l'écran                      | La règle est-elle enfouie dans React ?           | Extraire une fonction pure ou un petit composant.                            |

### 12.2 Complexité cognitive avant complexité cyclomatique

La complexité cyclomatique compte les chemins. Elle est utile, mais le lecteur souffre aussi de :

- changements fréquents de niveau d'abstraction ;
- noms qui obligent à mémoriser leur signification ;
- dépendances implicites entre états ;
- effets déclenchés à distance ;
- contrats non validés ;
- navigation entre trop de micro-fichiers.

#### Réflexe pratique

Chronométrez mentalement la compréhension : “Puis-je expliquer cette fonction en une phrase exacte après une lecture ?” Si la phrase contient plusieurs “et puis”, la responsabilité mérite un examen.

## 13 Refactoriser sans tout réécrire

### 13.1 Principe : préserver le comportement

Une réécriture totale remplace des défauts connus par des défauts inconnus. La méthode la plus sûre consiste à rendre un petit comportement observable, le déplacer, vérifier, puis recommencer.

Source : *Clean Code*, 2e éd., chap. 18, tests et refactoring, p. 513–521.

### 13.2 Méthode en huit étapes

1. **Choisir un parcours concret.** Par exemple : “créer une entrée puis la voir apparaître”.
2. **Écrire un filet de sécurité.** Un test de règle, de contrat API ou un test de parcours critique. Si aucun test n’est disponible, noter précisément le comportement manuel attendu.
3. **Renommer sans déplacer.** Remplacer `data`, `handle` et les nombres magiques. Le diff reste facile à relire.
4. **Extraire le domaine pur.** Déplacer calculs, sélections et validation hors de React, avec tests.
5. **Créer la frontière API.** Centraliser endpoint, statut HTTP, parsing et mapping DTO.
6. **Extraire l’orchestration.** Introduire un hook seulement si l’état React reste complexe après les extractions précédentes.
7. **Découper le JSX.** Extraire les unités visuelles nommables avec des props simples.
8. **Nettoyer les dépendances.** Supprimer imports inutiles, code mort et abstractions devenues sans usage ; exécuter tests, lint et build.

### 13.3 Taille des lots

Un bon lot de refactorisation :

- conserve le comportement utilisateur ;
- peut être relu en quelques minutes ;
- possède un objectif unique, par exemple “extraire le calcul de score” ;
- laisse l’application exécutable ;
- n’impose pas simultanément un renommage massif, une nouvelle bibliothèque et une nouvelle architecture.

#### Erreur fréquente

Déplacer tous les fichiers avant d’améliorer leurs responsabilités. Une arborescence propre ne répare pas des dépendances confuses ; elle peut seulement les rendre plus difficiles à retrouver.

### 13.4 Ordre de priorité

1. Défauts et risques de données.
2. Contrats externes non validés et erreurs masquées.
3. Règles métier dupliquées.
4. Composants qui bloquent chaque modification.
5. Nommage et placement.
6. Esthétique locale sans impact.

#### À retenir

Refactoriser signifie changer la structure sans changer le comportement observable. Chaque étape doit réduire un risque identifiable et rester vérifiable.

## 14 Checklists de décision

## 14.1 Avant de créer un composant

- ☐ Puis-je lui donner un nom visuel ou métier précis ?
- ☐ Possède-t-il une responsabilité différente de son parent ?
- ☐ Ses props peuvent-elles rester petites et explicites ?
- ☐ Cache-t-il une vraie complexité ou seulement trois balises ?
- ☐ Son état est-il local à cette partie de l'interface ?
- ☐ Est-ce une réutilisation réelle ou seulement anticipée ?
- ☐ Le découpage améliorera-t-il les tests ou la lecture ?

## 14.2 Avant de créer une fonction

- ☐ Le nom décrit-il une action et un résultat ?
- ☐ La fonction reste-t-elle à un niveau d'abstraction ?
- ☐ Ses arguments forment-ils un concept à nommer ?
- ☐ Produit-elle un résultat ou déclenche-t-elle un effet clairement annoncé ?
- ☐ Les erreurs possibles sont-elles visibles dans le contrat ?
- ☐ Peut-elle être pure ?
- ☐ L'extraction évite-t-elle une duplication conceptuelle ?
- ☐ Si elle dépasse 40 lignes, pourquoi reste-t-elle cohérente ?

## 14.3 Avant de créer un fichier

- ☐ Quelle feature possède ce fichier ?
- ☐ Quel rôle unique son dossier exprime-t-il ?
- ☐ Son nom correspond-il à son export principal ?
- ☐ Doit-il réellement être partagé ?
- ☐ Quelles couches peuvent l'importer ?
- ☐ Risque-t-il de créer un cycle ?
- ☐ Un fichier existant possède-t-il déjà cette responsabilité ?

## 14.4 Avant une fusion

- ☐ Les noms emploient-ils le vocabulaire du domaine ?
- ☐ Les états vide, chargement, erreur et succès sont-ils distincts ?
- ☐ Les réponses externes sont-elles validées ?
- ☐ Aucun secret n'est-il exposé dans le bundle client ?
- ☐ Les règles métier importantes sont-elles testées ?
- ☐ Les imports respectent-ils la direction des dépendances ?
- ☐ Le lint, le contrôle TypeScript, les tests et le build passent-ils ?
- ☐ Le changement est-il assez petit pour être expliqué simplement ?

# 15 Exercices de consolidation

### Exercice 4 – Réécrire une condition

Améliorez :

```
if (user && user.settings && user.settings.tracking === true) {  
  // ...  
}
```

Le code doit exprimer le concept métier et gérer l'absence de valeur.

### Correction 4

```
function canTrackDays(user: User | null): boolean {  
  return user?.settings.trackingEnabled === true;  
}  
  
if (canTrackDays(user)) {  
  // ...  
}
```

La fonction n'est utile que si ce concept est répété, complexe ou important. Sinon, `user?.settings.trackingEnabled === true` peut rester local.

### Exercice 5 – Trouver les couches

Classez les éléments suivants : `EnergyGauge`, `fetchDayEntries`, `calculateStreak`, `useDayEntries`, `DayEntryDto`.

### Correction 5

`EnergyGauge` : UI ; `fetchDayEntries` : infrastructure/API ; `calculateStreak` : domaine ; `useDayEntries` : application/UI React ; `DayEntryDto` : frontière API.

### Exercice 6 – Décider d'une extraction

Deux composants formatent une date avec la même expression. L'un affiche une date longue en français, l'autre une date compacte dans un graphique. Faut-il créer immédiatement `formatDate` ?

### Correction 6

Non, pas sous ce nom générique. Les intentions diffèrent et peuvent évoluer séparément. Créez éventuellement `formatEntryDateLabel` et `formatChartDateTick`, ou conservez les expressions localement jusqu'à ce que le besoin commun soit prouvé.

### Exercice 7 – État impossible

Pourquoi `isLoading`, `isError`, `data` et `errorMessage` comme états indépendants posent-ils problème ? Proposez une alternative.

### Correction 7

Ils permettent des combinaisons contradictoires. Utilisez une union discriminée `loading | success(data) | error(message)`. Chaque état porte uniquement les données qui lui appartiennent.

### Exercice 8 – Refactorisation sûre

Votre composant de 350 lignes charge, filtre, trie et affiche les entrées. Quelle est la première extraction recommandée ?

### Correction 8

Après avoir sécurisé le comportement, commencez par une règle pure facile à nommer et tester, par exemple `selectVisibleEntries(entries, filters)`. Ne déplacez pas d'abord tout le JSX ou tous les fichiers.

## 16 Rappel actif et plan de progression

### 16.1 Questions à reconstruire sans le cours

1. Quelle différence faites-vous entre responsabilité et petite taille ?
2. Pourquoi une réponse API typée en TypeScript reste-t-elle non fiable ?
3. Quand un hook personnalisé est-il préférable à une fonction pure ?
4. Pourquoi `utils.ts` devient-il souvent un problème ?
5. Que signifie “la page lit comme une table des matières” ?
6. Quelles fonctions Next.js disparaissent lors d’un export statique ?
7. Comment distinguer donnée vide et échec de chargement ?
8. Pourquoi 40 lignes constituent-elles un signal et non une loi ?

### 16.2 Calendrier de révision

| Moment        | Travail                                                                                   |
|---------------|-------------------------------------------------------------------------------------------|
| Aujourd’hui   | Refaire les exercices 1 à 3 sans correction.                                              |
| Dans 1 jour   | Expliquer oralement les quatre couches et dessiner l’arborescence de mémoire.             |
| Dans 3 jours  | Prendre une fonction réelle et produire un diagnostic basé sur les signaux de complexité. |
| Dans 7 jours  | Refactoriser un calcul pur et ajouter son test.                                           |
| Dans 14 jours | Revoir un appel API réel : statut, parsing, DTO, erreur et état UI.                       |
| Dans 30 jours | Réauditer un parcours complet avec la checklist finale.                                   |

## 17 Checklist finale de développement

### Lisibilité locale

- ☐ Les noms révèlent le domaine, l’unité et l’état.
- ☐ Les nombres et chaînes métier importants sont nommés.
- ☐ Chaque fonction peut être résumée en une phrase précise.
- ☐ Les niveaux d’abstraction ne sont pas mélangés.
- ☐ Les commentaires expliquent des raisons, contraintes ou risques.
- ☐ Les branches invalides sortent tôt et restent lisibles.

### React et TypeScript

- ☐ Le JSX ne contient pas de règle métier complexe.
- ☐ Les valeurs dérivées ne sont pas stockées inutilement dans l’état.
- ☐ Les effets synchronisent avec l’extérieur ; ils ne servent pas de pipeline général.
- ☐ Les états asynchrones impossibles ne sont pas représentables.
- ☐ Les hooks encapsulent une mécanique React réelle.
- ☐ `unknown` est validé aux frontières ; `any` est évité.

### Architecture et API

- ☐ Chaque fichier a une feature propriétaire et un rôle identifiable.
- ☐ Le domaine n’importe ni React, ni Next.js, ni HTTP.
- ☐ Les DTO sont convertis à la frontière.
- ☐ Les erreurs réseau, HTTP et de format sont distinguées.
- ☐ Les imports vont vers des couches plus stables.
- ☐ L’export statique n’utilise pas de capacité serveur à l’exécution.

- ☐ Aucun secret n'est intégré dans une variable publique.

### Changement sûr

- ☐ Le comportement à préserver est écrit.
- ☐ Les règles et frontières critiques sont testées.
- ☐ Le refactoring est séparé autant que possible du changement fonctionnel.
- ☐ Le lot est petit, relisible et réversible.
- ☐ TypeScript, lint, tests et build passent.
- ☐ La structure est plus simple à expliquer après le changement.

## 18 Ce qu'une analyse du dépôt réel doit confirmer

Ce cours propose une architecture réaliste, mais ne peut pas décider sans le code :

- si le projet utilise le routeur `app/` ou `pages/` ;
- si une bibliothèque de requêtes, de formulaires ou de validation est déjà présente ;
- quels composants sont réellement trop volumineux ;
- quelles règles métier sont dupliquées ;
- quels dossiers `constants/` et `static/` contiennent ;
- quelles conventions d'imports, tests et styles sont déjà établies ;
- quel est le contrat exact du backend et son modèle d'authentification ;
- quelles capacités de Next.js sont compatibles avec la configuration d'export actuelle.

La prochaine étape rationnelle n'est donc pas de déplacer tout le dépôt vers cette arborescence. C'est d'auditer un parcours utilisateur, dessiner ses dépendances réelles, puis appliquer la méthode progressive du chapitre 13.

## 19 Références utilisées

1. Robert C. Martin, *Clean Code: A Handbook of Agile Software Craftsmanship*, deuxième édition. Chapitre 4, noms significatifs, PDF p. 140–147.
2. Ibid., chapitre 5, commentaires, PDF p. 161–172.
3. Ibid., chapitre 6, formatage, PDF p. 198–200.
4. Ibid., chapitres 7, 8 et 10, fonctions, arguments, erreurs et extraction, PDF p. 228–269 et 334–336.
5. Ibid., chapitres 12 et 13, objets, structures de données, cohésion et classes, PDF p. 399–420.
6. Ibid., chapitres 14 et 15, discipline et lisibilité des tests, PDF p. 466–480.
7. Ibid., chapitres 18 et 19, conception simple, duplication, YAGNI et responsabilité unique, PDF p. 513–528.
8. Ibid., chapitres 24 à 27, frontières architecturales et règle de dépendance, PDF p. 691–724.

La propreté du code n'est pas un état final. C'est la capacité à rendre le prochain changement plus local, plus explicite et plus vérifiable.